

User Account Enumeration

Timing differential in the password reset flow

Christian Bai | Redacted work sample

LOW

A finding from a client penetration test, published as a writing sample. The client, the application, all hostnames, cookie names, tokens, account values and dates have been removed or replaced with placeholders. Timing values and the reasoning are unchanged.

Summary

The password reset endpoint takes measurably longer to respond when the submitted email address belongs to a real account than when it does not. The status code, the redirect, and the visible response body are identical in both cases, so the application looks correct from the browser. The difference is in how long the server takes.

An attacker who can tell those two cases apart can confirm whether any given email address has an account, and can automate that across a list.

Why the usual check misses it

The standard test for account enumeration is to compare response content. Does the error message differ, does the status code differ, does the redirect differ. This application passes all three. Someone testing by eye would record it as not vulnerable and move on.

The signal is in the timing, and in this case the application reported its own timing in a response header, so no measurement was required.

Evidence

All hosts, tokens, and account values below are placeholders. Timing values are as observed.

Valid account

```
POST /users/password HTTP/2
Host: app.example.com
Cookie: SessionId=[redacted]
Content-Type: application/x-www-form-urlencoded

authenticity_token=[redacted]&user[login_email]=[valid test account]
&company=[redacted]&commit=Send+reset+instructions
```

```
HTTP/2 302 Found
Location: https://app.example.com/users/sign_in
Cache-Control: no-store
X-Runtime: 1.190181
```

Server processing time: **1190.181 ms**

Control, address with no account

The control reused the same form session, endpoint, company value, and request structure. Only the email address changed.

```
POST /users/password HTTP/2
Host: app.example.com
Cookie: SessionId=[redacted]
Content-Type: application/x-www-form-urlencoded

authenticity_token=[redacted]&user[login_email]=[address with no account]
&company=[redacted]&commit=Send+reset+instructions
```

```
HTTP/2 302 Found
Location: https://app.example.com/users/sign_in
Cache-Control: no-store
X-Runtime: 0.019307
```

Server processing time: **19.307 ms**

What the two have in common

Same status. Same redirect target. Same visible content. Nothing in the response body discloses whether the account exists.

The valid-account requests were consistently slower than the controls across the requests sent, though the sample was bounded and no statistical claim is made beyond that. The behaviour is consistent with the server preparing and sending the reset email before returning its response, which only happens when there is an account to send to.

Impact

Phishing and password spraying both need valid usernames to aim at. A confirmed list of account holders makes both cheaper.

The severity is held at Low for a specific reason. Enumerating through this endpoint sends a real password reset email to every valid address it finds, so a run of any size generates a wave of unexpected reset messages and a help desk that notices. The vulnerability is real and the practical cost of exploiting it quietly is high.

Recommendation

Return the response in the same timeframe regardless of whether the address matches an account. The reset email should be queued rather than sent synchronously, so the response does not wait on it.

Rate limiting or a CAPTCHA on the endpoint limits automated attempts. If a CAPTCHA is not required on every request, trigger it on repeated attempts keyed to source address rather than to the current session, since an attacker running this at scale is not reusing a session.

References

OWASP, Testing for Account Enumeration and Guessable User Account

PortSwigger, Preventing username enumeration